

Multi-RAT

Project Overview — Bachelorprojekt

Dette dokument giver et samlet overblik over Multi-RAT projektet: visionen, arkitekturen, dataflowet, hvad der er færdigt og hvad der mangler. Det er tænkt som det første dokument du læser når du vender tilbage til projektet efter en pause.

Forfatter	Viktor Munk
Institut	DTU — Cybersecurity, 4. semester
Kursus	Bachelorprojekt + Intro to Digital Communication
Sidst opdateret	2026-06-01
Midterm deadline	2026-06-03
Final deadline	2026-06-24

1. Visionen

Multi-RAT (Multiple Radio Access Technology) er et system, der sender datapakker over flere trådløse radioteknologier **samtidigt**, så tab eller dårlig dækning på den ene sti kompenseres af den anden. I vores prototype er det **Wi-Fi** og **5G/LTE** på en Android-telefon.

Konkret mål:

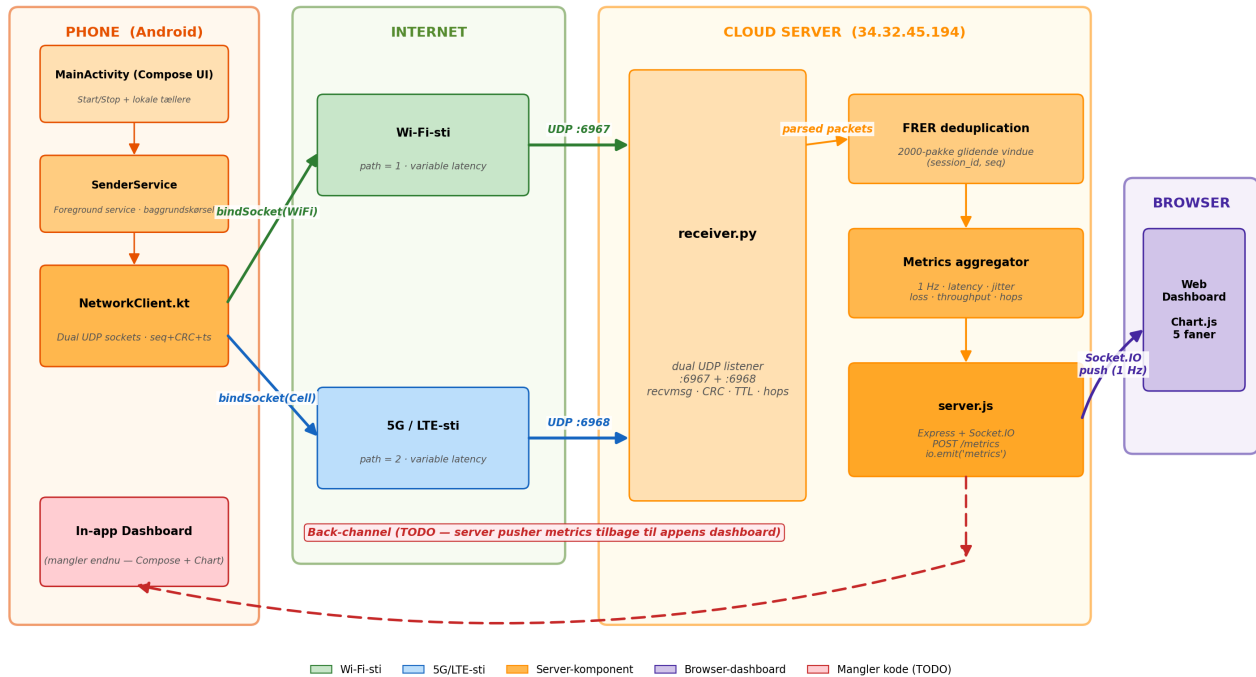
- Mobil-app sender pakker over begge radioer parallelt (1+1 protection-skema).
- Server modtager begge kopier, fjerner duplikater (IEEE 802.1CB FRER-stil).
- Live-statistik vises i **browser-dashboard** (er kodet).
- Live-statistik vises også i **selve appen** (mangler endnu — back-channel server→mobil).
- Systemet kører 24/7 på Google Cloud, så det kan testes over rigtigt 5G.
- Senderen skal kunne udgives til Google Play, så andre kan teste fra deres telefon.

2. Arkitektur

Systemet består af tre lag: en Android-sender (Kotlin), en cloud-baseret modtager (Python + Node), og et browser-dashboard. Den røde stiplede pil i Figur 1 er back-channel'en til in-app dashboard'et, som endnu mangler kode.

Figur 1 — Multi-RAT systemarkitektur

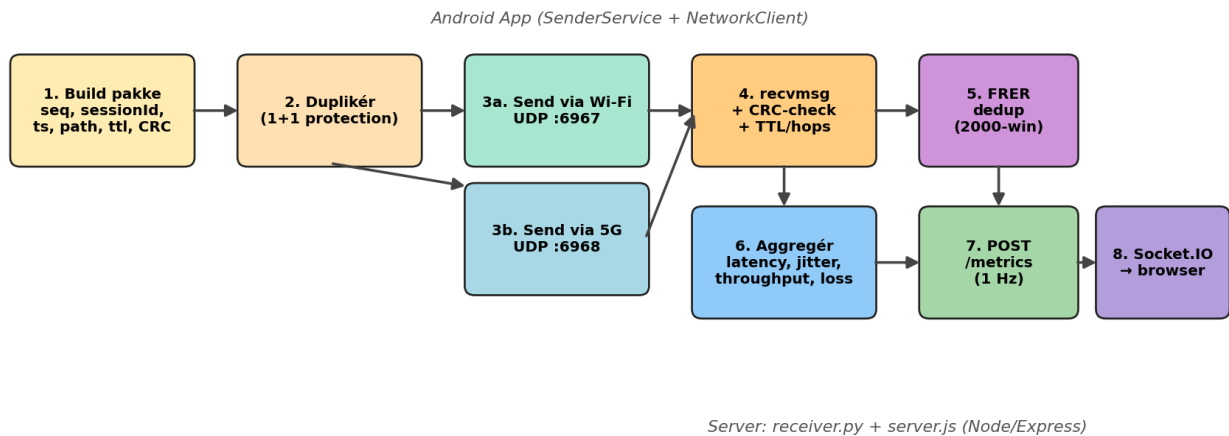
1+1 protection: én pakke bygges, dupleres og sendes parallelt over Wi-Fi og 5G. Serveren modtager begge kopier, fjerner duplikater (FRER) og pusher metrics til dashboardet.



Figur 1. Android-appen åbner to UDP-sockets — en bundet til Wi-Fi-interfacet og en bundet til celle-radioen via `ConnectivityManager.bindSocket()`. Begge sockets sender til samme server, men på forskellige porte (6967/6968), så modtageren kan skelne stierne.

3. Pakkens vej gennem systemet

Figur 2 — Pakkens vej fra mobil til dashboard

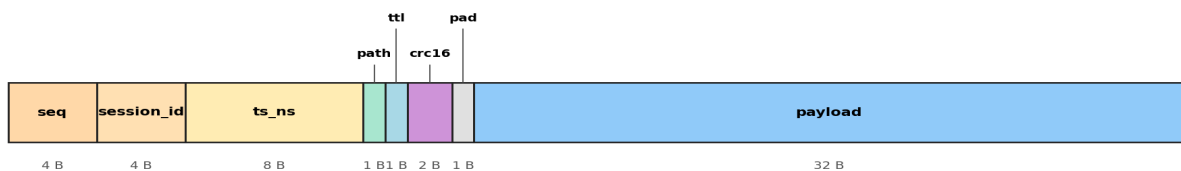


Figur 2. Hver pakke bygges én gang på telefonen, dupleres og sendes via begge radioer. Serveren modtager begge kopier, validerer CRC, beregner TTL-hops, og kører FRER-deduplikering i et glidende vindue på 2000 pakker. Hver sekund POST'es en aggregeret metric-snapshot til Express, der pusher den videre til browseren via Socket.IO.

4. UDP-pakkeformatet

Figur 3 — UDP-pakkeformat (HDR_FMT = !IIQBBHx)

Total = 53 bytes (21 B header + 32 B payload)



Figur 3. Format-streng !IIQBBHx i Python / ByteBuffer .BIG_ENDIAN i Kotlin. Payload er 32 B konstant. Header-overhead = $21/53 \approx 39,6\%$, så ved 20 pkt/s = 8 480 bit/s rå transmissionsrate pr. sti, hvor af 5 120 bit/s er nyttig last.

5. Hvad er færdigt — og hvad mangler

Komponent	Status	Note
Android UDP-sender (dual radio)	✓ Færdig	NetworkClient.kt
Foreground service (baggrundskørsel)	✓ Færdig	SenderService.kt
Compose UI (start/stop + lokale tællere)	✓ Færdig	MainActivity.kt
Python receiver (dual-port, CRC, hops)	✓ Færdig	receiver.py
FRER deduplikering	✓ Færdig	2000-pakke vindue
Express + Socket.IO server	✓ Færdig	server.js
Web-dashboard (5 faner, Chart.js)	✓ Færdig	public/index.html

Google Cloud deployment	✓ Kører	34.32.45.194:3000
Back-channel server → mobil-app	✗ Mangler	In-app dashboard
In-app dashboard (Chart i Compose)	✗ Mangler	Skal kodes
QUIC-transport	~ Eksperimentel	quic_implementation/
Autentificering af afsender	✗ Mangler	Sikkerheds-risiko
Google Play release	✗ Ikke endnu	Fremtid

6. Serveradgang (Google Cloud VM)

Server-VM'en kører 24/7 i Google Cloud uafhængigt af om du er logget ind — dashboardet er offentligt tilgængeligt på `http://34.32.45.194:3000`. SSH-adgangen bruges kun når du skal **ind på maskinen** for at se logs, deploye en ny version, eller starte servicen.

SSH-login:

```
ssh -i ~/.ssh/id_ed25519 viktormunk@34.32.45.194
```

Vigtig forudsætning:

`node server.js` skal køre som en **baggrundsproces** der overlever logout — typisk via `pm2`, `systemd` eller `tmux`. Hvis I blot har kørt `node server.js` direkte i SSH-sessionen, stopper serveren når SSH-vinduet lukkes. Tjek status med fx `pm2 status` eller `systemctl status multi-rat`.

Sundhedstjek udefra (uden SSH):

`curl http://34.32.45.194:3000/health` — viser uptime, alder på seneste metric-snapshot, og receiver-process-status. Dette er den hurtigste måde at se om systemet kører.

7. Næste skridt

Midterm-rapport. Aflever senest 2026-06-03. Tilføj evt. in-app dashboard og sikkerhed i Future Work.

Back-channel. Design protokol server → mobil. Mulighed: WebSocket fra serveren, eller serveren POST'er til en lille endpoint i appen.

In-app dashboard. Tilføj Compose-skærm med graf for latency/jitter/loss baseret på back-channel data.

Sikkerhed. Mindst HMAC over header — ellers kan enhver spamme serveren. Nævn i threat-model.

QUIC-integration. Skift UDP-laget ud med QUIC og sammenlign latency/loss-karakteristik.

Final rapport. Deadline 2026-06-24. Demonstration 2026-06-26.